RANGEN V2 System Architecture

1. High-Level Architecture

RANGEN V2 follows a **Hybrid Intelligence Architecture**, integrating Deep Learning (DL) for semantic understanding and Machine Learning (ML) for system decision-making. It combines modern Agentic patterns (ReAct, Self-Reflection) with a robust Neural Infrastructure.

```
graph TD
    User[User / Client] --> UI[Presentation Layer (Streamlit)]
    User --> API[API Layer (FastAPI)]

    subgraph "Neural Services (Local DL)"
        NeuralRouter[Neural Router (Zero-Shot DL)]
        NeuralRerank[Neural Reranker (Cross-Encoder)]
    end

    subgraph "Core System"
        API --> Coordinator[Execution Coordinator]
        UI --> Coordinator

        Coordinator --> NeuralRouter
        NeuralRouter -->|Intent| Workflow[Workflow Engine (LangGraph)]

        Workflow --> Reasoning[Reasoning Agent (ReAct)]
        Workflow --> Quality[Quality Evaluator]
        Workflow --> Direct[Direct Executor]
    end

    subgraph "Infrastructure"
```

```
        Reasoning --> LLM[LLM Integration Service]
        Reasoning --> Tools[Tool Registry]
        Tools --> NeuralRerank
        NeuralRerank --> KMS[Knowledge Management System]

        Monitor[Performance Monitor (ML-Ready)] -.-> Core System
        Redis[(Redis Cache)] -.-> Context[Context Manager]
    end
```

2. Core Components

2.1 Neural Infrastructure (`src/core/neural/`)

The "Brain" of the system, running local DL models for high-precision decision making. - **Neural Router**: – **Model**: `facebook/bart-large-mnli` (Zero-Shot Classification). - **Function**: Classifies query intent (Research, Chat, Coding) without rule-based heuristics. - **Advantage**: Understands semantic nuance (e.g., distinguishing "write code" from "explain code"). - **Neural Reranker**: – **Model**: `cross-encoder/ms-marco-MiniLM-L-6-v2`. – **Function**: Re-scores retrieval results by analyzing deep query-document interaction. - **Advantage**: Significantly higher relevance than standard Cosine Similarity. - **Factory Pattern**: Models are managed via `NeuralServiceFactory` for lazy loading and dependency injection.

2.2 Execution Coordinator (`src/core/execution_coordinator.py`)

The central orchestrator. - **Responsibilities**: Manages the global execution state via `LangGraph`. – **Workflow**: `NeuralRouter` –> `Agent` –> `Quality Evaluator` –> `Finalize`.

2.3 Smart Router (`src/core/configurable_router.py`)

Intelligent traffic control. - **Logic**: 1. **System Health**: Circuit breaker check. 2. **Neural Intent**: Uses DL model to classify intent with

confidence score. 3. **Fallback**: Degrades to heuristics if model confidence is low.

2.4 Reasoning Agent (`src/agents/reasoning_agent.py`)

The primary problem-solving entity. - **Pattern**: **ReAct (Reasoning + Acting)**. – **Features**: – **Prompt RAG**: Uses `PromptOrchestrator` to dynamically retrieve relevant few-shot examples via Neural Reranker. – **Dialectical Reasoning (DDL β≈2)**: Implements a "Challenge Node" that acts as a Devil's Advocate. Before finalizing an answer, the agent critiques its own reasoning to find biases or edge cases, ensuring robust conclusions. - **Tools**: Access to `RetrievalTool` (enhanced with Neural Reranker).

2.5 Context Manager (`src/core/context_manager.py`)

Manages session state with pluggable persistence (Redis/Memory). - **Features**: – **Persistence**: Supports memory and Redis backends. - **Smart Forgetting (DDL β≈1)**: Uses `NeuralReranker` to detect topic shifts. If a new query is semantically unrelated to recent history (score < β_threshold), it automatically archives old context. - **Context Summarization**: When archiving, generates a concise summary of the old context to maintain long-term memory.

2.6 Prompt Engineering (`src/services/prompt_engineering/`)

Advanced prompt construction system. - **PromptOrchestrator**: Dynamically assembles prompts by combining base templates, retrieved examples, and context summaries. - **Few-Shot Retrieval**: Uses `NeuralReranker` to find the most relevant reasoning examples from the example store.

3. AI Technology Stack

Deep Learning (DL) - *Semantic Understanding*

Component	Model / Technology	Role	Status
Intent Classification	BART-Large-MNLI (Zero-Shot)	Routing decision	✅ Active
Reranking	MS-MARCO-MiniLM (Cross-Encoder)	Retrieval precision	✅ Active
LLM Inference	DeepSeek / OpenAI	Core reasoning & generation	✅ Active

Machine Learning (ML) - *System Decision*

Component	Model / Technology	Role	Status
Anomaly Detection	Isolation Forest (Planned)	Health monitoring	⏳ Data Collection
Latency Prediction	Linear Regression (Planned)	Timeout adjustment	⏳ Data Collection

4. Optimization & Future Roadmap

4.1 Dynamic Parameter Learning (Planned)

- **Concept**: Transform hardcoded thresholds (e.g., Smart Forgetting β=0.2) into learnable parameters.
- **Mechanism**: Use Reinforcement Learning or simple heuristic adjustments based on user feedback (thumbs up/down).

4.2 Architecture Health Check (Planned)

- **Concept**: A dedicated module `HybridIntelligenceHealthCheck` to monitor the synergy between Neural components.
- **Function**: Auto-adjust weights (e.g., fallback to rules) if Neural Router accuracy drops.

4.3 Cost & Performance Analysis

- **Intent Classification**: Local BART (~$0.00001) vs GPT-4 API (~$0.002) –> **200x Cost Reduction**.
- **Retrieval**: Local Cross-Encoder vs API Rerank -> **20x Cost Reduction**.

5. Key Design Decisions

- **Hybrid Intelligence**: Combines LLMs (Generative) with specialized DL models (Discriminative) for optimal cost/performance.
- **Local Inference**: Neural Router and Reranker run locally (CPU/GPU), reducing API costs and latency.
- **DDL Philosophy**: System is designed as a dynamic organism with adjustable connections (β parameters) rather than a static graph.
- **LangGraph**: Enables cyclic workflows (Retry loops) and complex state management.
- **Pluggable Architecture**: Neural components are injected via interfaces (`IIntentClassifier`), allowing easy model swapping.

6. Directory Structure Map

- `src/core/neural/` : DL model implementations and factory.
- `src/core/nodes/` : Graph nodes (Router, Quality).
- `src/agents/` : Agent implementations.
- `src/prompts/` : Prompt templates.

- `src/services/` : Infrastructure services.
- `scripts/` : Operational scripts.